

Project: Resilient and Scalable Web Application Deployment on AWS

Version: 1.0

Name: Milan Rai

Table of Contents

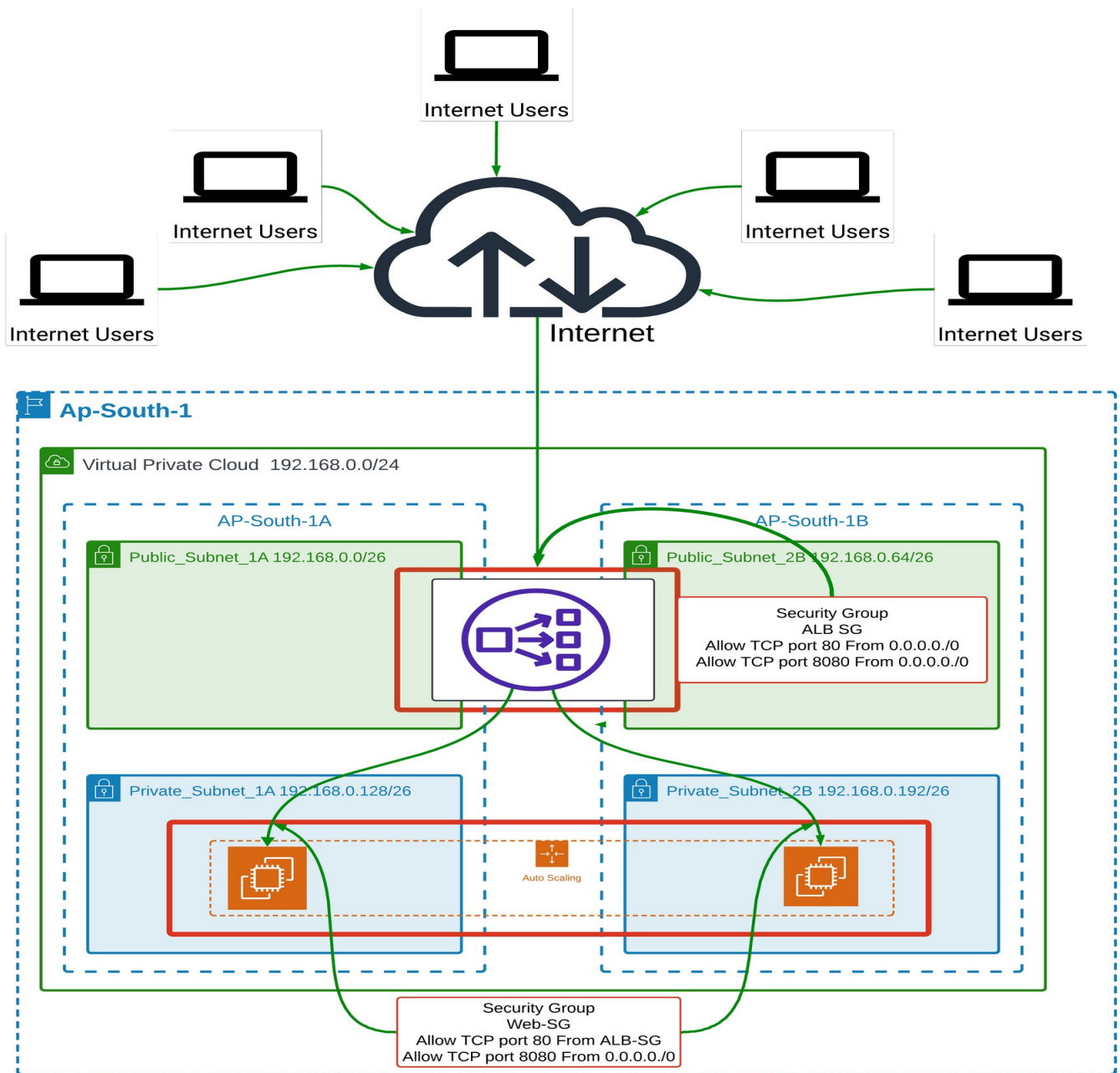
1. Introduction
2. Prerequisites
3. Architecture Overview
4. AWS Services Configuration 4.1 Virtual Private Cloud (VPC) 4.2 Elastic Compute Cloud (EC2) 4.3 Elastic File System (EFS) 4.4 Application Load Balancer (ALB) 4.5 Route 53
5. Application Deployment
6. Security Configuration

1. Introduction

- **Purpose:** This guide outlines the implementation steps to deploy a highly available and auto-scaling Python web application on AWS infrastructure.
- **Scope:** The guide covers VPC setup, EC2 Auto Scaling, ALB configuration, shared EFS storage, and DNS routing via Route 53 in the ap-south-1 region.

2. Prerequisites

- **AWS Account:** Active AWS Management Console access with administrative privileges.
- **IAM Roles:** IAM role configured with permissions for EC2 instances to access EFS and CloudWatch metrics.
- **3. Architecture Overview**

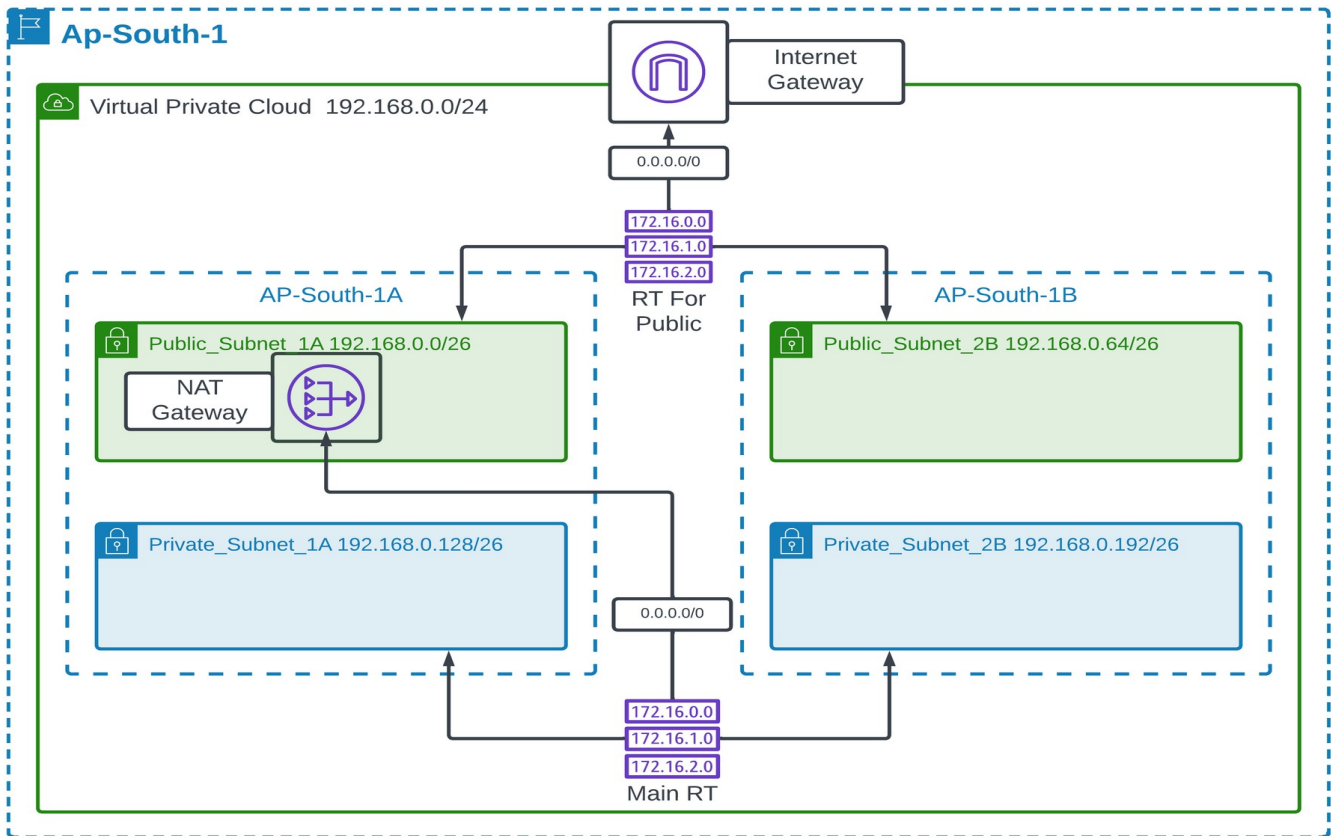


(Depicting the relationship between VPC, EFS, EC2 instances, Auto Scaling, ALB, and Route 53)

Description: The architecture routes traffic through Route 53 to an Application Load Balancer, which distributes requests across auto-scaling EC2 instances in private subnets, all sharing persistent data via EFS.

4. AWS Services Configuration

4.1 Virtual Private Cloud (VPC)



(Detailing the setup of the VPC, including public and private subnets across multiple AZs)

Step 1: Create the VPC

- Name tag: Ap-South-1
- IPv4 CIDR block: 192.168.0.0/24

Step 2: Create Subnets

- **Public Subnet 1A:** ap-south-1a | CIDR: 192.168.0.0/26
- **Private Subnet 1A:** ap-south-1a | CIDR: 192.168.0.128/26
- **Public Subnet 2B:** ap-south-1b | CIDR: 192.168.0.64/26
- **Private Subnet 2B:** ap-south-1b | CIDR: 192.168.0.192/26

Step 3: Setup Internet Gateway & NAT Gateway

- Created IGW attached to the VPC for public subnet internet access.
- Deployed a NAT Gateway in the public subnet to allow private instances outbound internet access.

Step 4: Create Route Tables

- Configured Main Route Table for private subnets pointing 0.0.0.0/0 to the NAT Gateway.

- Configured Public Route Table for public subnets pointing 0.0.0.0/0 to the Internet Gateway.

4.2 Elastic Compute Cloud (EC2)

- **Instance Setup:** Created a Launch Template using Amazon Linux 2023.
- **Auto Scaling:** Configured an Auto Scaling Group across the private subnets with Target Tracking policies based on CPU utilization.

4.3 Elastic File System (EFS)

- Created an EFS file system with mount targets in both private subnets to provide shared storage for the EC2 instances.

4.4 Application Load Balancer (ALB)

- Deployed an internet-facing ALB in the public subnets, routing incoming HTTP traffic to a target group containing the auto-scaled EC2 instances.

4.5 Route 53

- Configured a public hosted zone for the custom domain and created an Alias record pointing traffic to the ALB DNS name.

5. Application Deployment

- Deployed the application using a User Data shell script within the Launch Template. The script installs Python, Flask, EFS utilities, mounts the file system, and launches the application upon boot.

6. Security Configuration

- **Security Groups:**
 - ALB Security Group: Allows inbound HTTP (port 80) from 0.0.0.0/0.
 - EC2 Security Group: Allows inbound HTTP (port 80/8080) strictly from the ALB Security Group.
- **Data Encryption:** EFS data is encrypted at rest and in transit using TLS.